

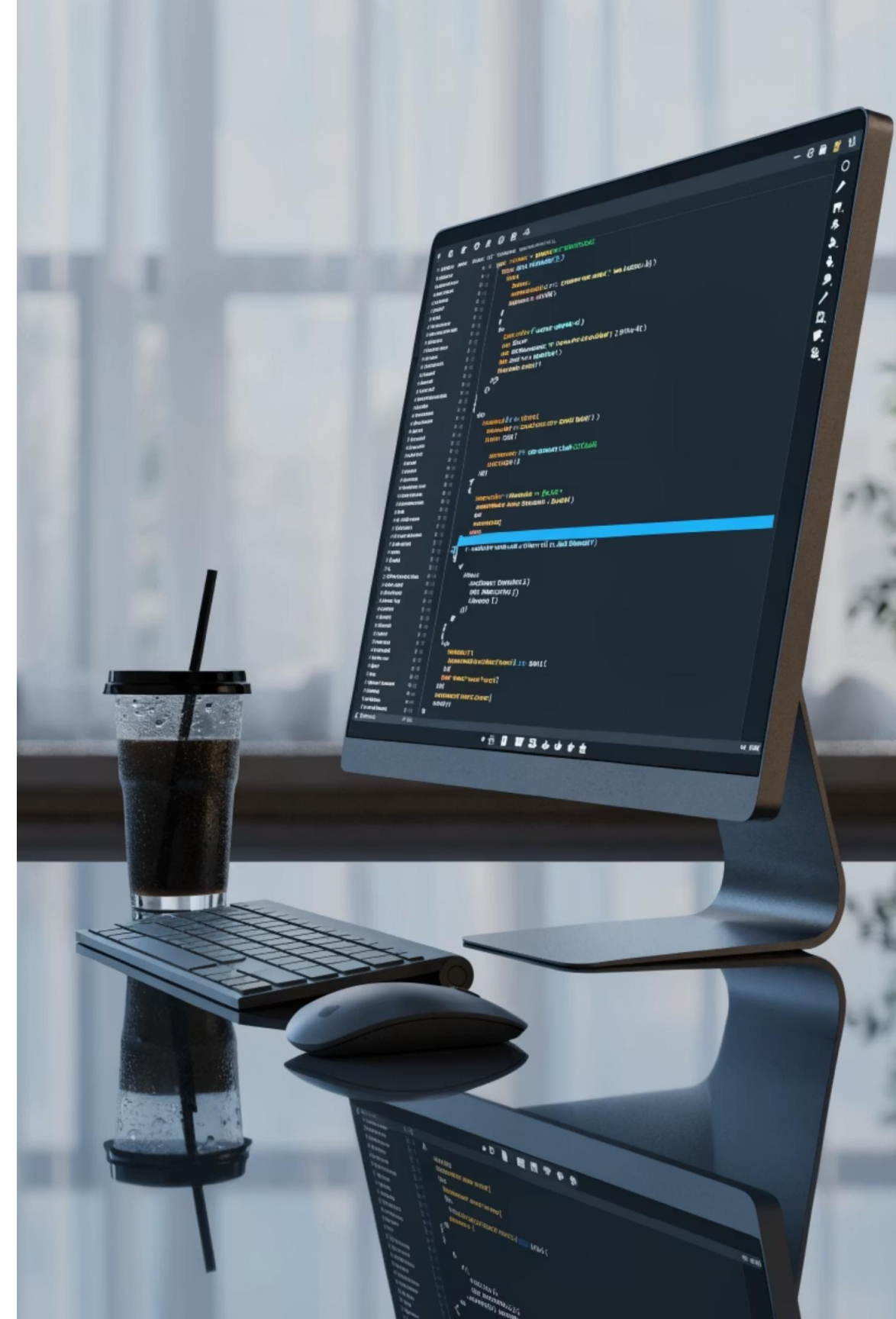
Estruturas de Controlo e Funções em Python

Técnico de Desenvolvimento de Software

UC 00618 – Criar aplicações em linguagem de programação Python

Formador: Ricardo Morgado

Cofinanciado por:



Objetivos da Aula

No final desta sessão, serás capaz de programar de forma estruturada e profissional. Aqui está o que vais dominar:



Operadores

Aritméticos, relacionais e lógicos para cálculos e comparações.



Condições

Estruturas if, elif, else e match-case para tomar decisões.



Ciclos

while e for para repetir operações de forma controlada.



Funções

Organizar o código com funções reutilizáveis e com retorno.



O Exercício Contínuo

Sistema de Gestão de Encomendas

Ao longo da aula, vamos construir um programa completo que evolui a cada conceito aprendido. Nada é isolado — cada estrutura nova entra diretamente no sistema.



Inserir produtos

Registar dados de encomendas



Calcular valores

Preços, quantidades e totais



Aplicar descontos

Condições para descontos automáticos



Repetir operações

Menu em ciclo até o utilizador sair



Funções

Organizar a lógica de forma limpa

Como Pensa um Programa Python?

A Analogia da Cozinha

Qualquer programa funciona como uma receita culinária:

- Input = os ingredientes que entram
- Processamento = cozinhar e transformar
- Output = o prato final servido

Todo o código que vamos escrever segue sempre esta ordem.

A Estrutura Universal



Exemplo: O Primeiro Bloco do Sistema

Vamos começar com o mais simples possível — pedir o nome do cliente e cumprimentá-lo. Este padrão entrada → processamento → saída repete-se ao longo de todo o programa.

```
# Entrada
nome = input("Introduz o teu nome: ")

# Processamento
saudacao = "Olá, " + nome + "!"

# Saída
print(saudacao)
```

Operadores: A Calculadora do Programa

Os operadores são as ferramentas matemáticas e lógicas do Python. Permitem calcular, comparar e combinar condições. Existem três famílias essenciais que vais usar constantemente no sistema de encomendas.

+ Aritméticos

Para calcular valores numéricos.

⚖ Relacionais

Para comparar dois valores.

🔗 Lógicos

Para combinar condições.

Operadores Aritméticos

São os operadores de cálculo matemático. Tal como numa calculadora, servem para somar, subtrair, multiplicar e dividir — com algumas operações extra muito úteis.

Operador	Significado	Exemplo	Resultado
+	Adição	$10 + 3$	13
-	Subtração	$10 - 3$	7
*	Multiplicação	$10 * 3$	30
/	Divisão real	$10 / 3$	3.33...
//	Divisão inteira	$10 // 3$	3
%	Resto da divisão	$10 \% 3$	1
**	Potência	$2 ** 3$	8



O operador % (módulo) é muito útil para verificar se um número é par ou ímpar: `numero % 2 == 0`

Cofinanciado por:

Operadores Relacionais

Os operadores relacionais comparam dois valores e devolvem sempre `True` ou `False`. Pensa neles como perguntas: "É maior?", "São iguais?". São a base de todas as decisões do programa.

Operador	Significado
<code>==</code>	Igual a
<code>!=</code>	Diferente de
<code>></code>	Maior que
<code><</code>	Menor que
<code>>=</code>	Maior ou igual a
<code><=</code>	Menor ou igual a

Exemplo no sistema:

```
total = 75
```

```
# Comparações
```

```
total > 50      # True
```

```
total == 100    # False
```

```
total != 0      # True
```

```
total >= 75     # True
```

📄 ⚠ Erro comum: usar `=` (atribuição) em vez de `==` (comparação)!

Operadores Lógicos

Os operadores lógicos permitem combinar várias condições numa só. São como a linguagem do dia a dia: "e", "ou", "não".

and — "e"

Só é `True` se ambas as condições forem verdadeiras. Ex: *"Tem desconto de sócio e a compra é superior a 50€"*

```
total > 50 and socio == True
```

or — "ou"

É `True` se pelo menos uma condição for verdadeira. Ex: *"Tem desconto se for sócio ou se comprar mais de 100€"*

```
socio == True or total > 100
```

not — "não"

Inverte o valor lógico. `True` torna-se `False` e vice-versa. Ex: *"Aplica taxa se não for sócio"*

```
not socio
```



Operadores no Sistema de Encomendas

Vamos aplicar os três tipos de operadores diretamente no nosso sistema. Observa como calcular, comparar e combinar funcionam juntos:

```
# Dados da encomenda
preco = float(input("Preço unitário: "))
quantidade = int(input("Quantidade: "))

# Operador aritmético – calcular total
total = preco * quantidade

# Operador relacional – verificar desconto
tem_desconto = total > 50

# Operador lógico – condição combinada
print("Tem desconto:", tem_desconto)
print("Total calculado:", total)
```

Exercício: Calcular o Total de uma Compra

- 1 Pedir o nome do produto ao utilizador
Usa `input()` para recolher o nome.
- 2 Pedir o preço e a quantidade
Converte para `float` e `int` respetivamente.
- 3 Calcular o total com operador aritmético
`total = preco * quantidade`
- 4 Mostrar o resultado formatado
Usa `print()` com uma mensagem clara.



PARTE 3 — ESTRUTURAS CONDICIONAIS

Tomar Decisões com if / elif / else

A Analogia do Dia a Dia

"Se tiver dinheiro suficiente, compro. Senão, não compro."

Os programas tomam decisões exatamente assim. Com base numa condição, o código segue um caminho ou outro.

Sem condições, o programa seria sempre previsível — nunca reagiria a situações diferentes.

A estrutura base:

```
if condição:  
    # executa se True  
elif outra_condição:  
    # executa se a 2ª for True  
else:  
    # executa se nenhuma for True
```

  Não esquecer os dois pontos (:) no final de cada linha de condição!

if — A Decisão Simples

O `if` é a estrutura mais básica de decisão. O bloco de código dentro do `if` só é executado se a condição for verdadeira (`True`). Se for falsa, simplesmente é ignorado.

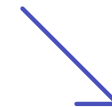
```
total = 75

if total > 50:
    print("Desconto disponível para esta encomenda!")
```



Condição Verdadeira

O bloco indentado é executado. O programa "entra" no `if`.



Condição Falsa

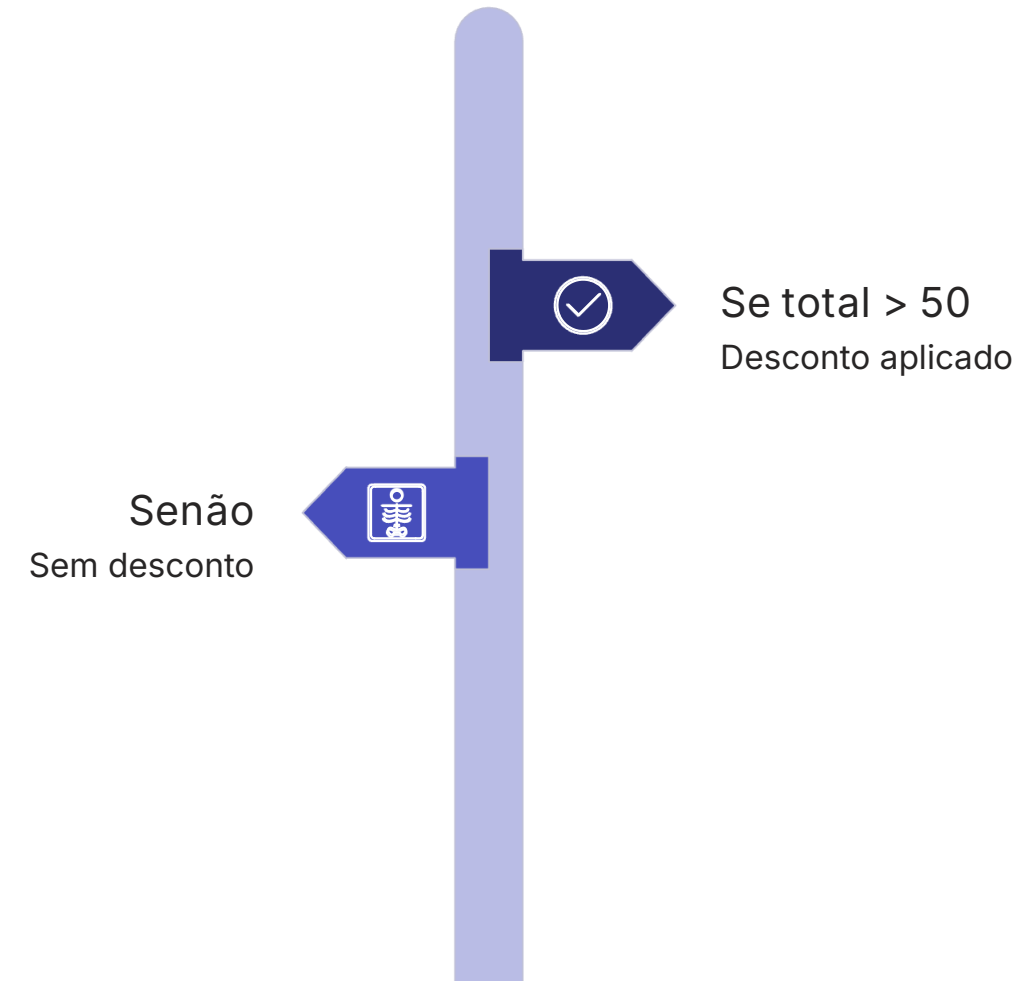
O bloco é ignorado. O programa continua depois do `if`.

if / else — Decisão Binária

Quando queremos um comportamento para cada caso — tanto quando a condição é verdadeira como quando é falsa — usamos o `else`. É uma decisão de dois caminhos.

```
total = 30

if total > 50:
    print("✅ Desconto de 10% aplicado!")
else:
    print("❌ Total insuficiente para
desconto.")
```



O programa sempre segue um dos dois caminhos — nunca ambos, nunca nenhum.

Cofinanciado por:

elif — Múltiplas Condições

Quando existem mais de dois cenários possíveis, usamos `elif` (abreviatura de "else if"). O Python verifica as condições pela ordem em que aparecem e executa apenas a primeira que for verdadeira.

```
total = 120

if total > 100:
    print("🏆 Desconto especial de 15%!")
elif total > 50:
    print("👍 Desconto de 10% aplicado.")
else:
    print("Sem desconto. Total abaixo de 50€.")
```

💡 A ordem importa! Coloca sempre as condições mais restritivas primeiro. Se puseres `total > 50` antes de `total > 100`, o segundo nunca será alcançado quando `total > 100`.

Aplicação: Descontos no Sistema

Vamos integrar as estruturas condicionais no nosso sistema de encomendas. A lógica de descontos é uma das funcionalidades mais importantes de qualquer sistema comercial.

```
preco = float(input("Preço unitário: "))
quantidade = int(input("Quantidade: "))
total = preco * quantidade

print("Total antes do desconto:", total, "€")

if total > 100:
    total_final = total * 0.85
    print("🏆 Desconto de 15% aplicado!")
elif total > 50:
    total_final = total * 0.90
    print("👍 Desconto de 10% aplicado.")
else:
    total_final = total
    print("Sem desconto.")
print("Total final:", total_final, "€")
```

❏ ? "Se o total for exatamente 50€, qual é o desconto? E se for 101€?" — Testar casos limite é fundamental.



match-case — O Menu de Opções

Disponível a partir do Python 3.10, o `match-case` é uma alternativa elegante ao encadeamento de múltiplos `elif`. É ideal para menus e seleção de opções.

Sintaxe do match-case

Estrutura base:

```
opcao = 1

match opcao:
    case 1:
        print("Nova encomenda")
    case 2:
        print("Ver encomendas")
    case 0:
        print("Sair")
    case _:
        print("Opção inválida")
```

Notas importantes:

- O **case _:** é o "caso por defeito"
Equivalente ao **else** — captura qualquer valor não listado.
- Cada **case** é mutuamente exclusivo
Apenas um bloco é executado por vez.
- Mais legível que muitos elif
Para 3 ou mais opções, o match-case é mais claro.

O Menu do Sistema de Encomendas

Usamos o `match-case` para criar a interface principal do nosso sistema. O utilizador escolhe uma opção e o programa age em conformidade.

```
opcao = int(input("\nEscolhe uma opção:\n1 - Nova encomenda\n2 - Ver produtos\n0 - Sair\n> "))

match opcao:
    case 1:
        print("📦 A registar nova encomenda...")
    case 2:
        print("📋 A listar produtos...")
    case 0:
        print("👋 A sair do sistema. Até logo!")
    case _:
        print("❌ Opção inválida. Tenta de novo.")
```

💡 Este menu vai ser a "espinha dorsal" do sistema. Mais à frente, vamos pô-lo a repetir automaticamente com o ciclo `while`.

Ciclos: Repetir sem Repetir Código

Na programação, muitas vezes precisamos de repetir operações — mostrar um menu várias vezes, percorrer uma lista de produtos, processar múltiplas encomendas. Os ciclos fazem isso de forma automática e eficiente.

while

Repete enquanto uma condição for verdadeira. Usa-se quando não sabemos quantas repetições haverá.

for

Repete para cada elemento de uma sequência. Usa-se quando sabemos o número de repetições.



O Ciclo while

O `while` repete um bloco de código enquanto a condição for verdadeira. Quando a condição se torna falsa, o ciclo termina e o programa continua.

```
opcao = -1

while opcao != 0:
    print("\n1 - Nova encomenda")
    print("2 - Ver produtos")
    print("0 - Sair")
    opcao = int(input("Opção: "))
    print("Escolheste:", opcao)

print("Programa terminado.")
```



Analogia: *"Enquanto a loja estiver aberta, continuo a atender clientes. Quando fechar, paro."*



Cuidado com o loop infinito: se a condição nunca se tornar falsa, o programa nunca para!

Quando Usar o while?

✓ Usa while quando...

- Não sabes quantas repetições vão acontecer
- O utilizador decide quando parar
- Dependes de uma condição que muda durante a execução
- Queres criar um menu que se repete

✗ Erros comuns com while:

- Esquecer de atualizar a variável de controlo
O ciclo nunca termina → loop infinito.
- Condição sempre verdadeira
Ex: `while True:` sem `break` → programa trava.
- Condição nunca verdadeira inicialmente
O ciclo não executa nenhuma vez.

O Ciclo for

O `for` percorre cada elemento de uma sequência — uma lista, uma string, ou um intervalo de números com `range()`. É ideal quando sabemos exatamente o que queremos percorrer.

```
# Percorrer um intervalo de números
for numero in range(5):
    print("Número:", numero)
```

Saída: 0, 1, 2, 3, 4

`range(5)`

Gera: 0, 1, 2, 3, 4

`range(1, 6)`

Gera: 1, 2, 3, 4, 5

`range(0, 10, 2)`

Gera: 0, 2, 4, 6, 8

📄 🧠 Analogia: "É como o professor a chamar os alunos da lista — percorre nome a nome, do primeiro ao último."

Cofinanciado por:

for com Listas

O `for` é especialmente poderoso com listas. Permite percorrer todos os elementos sem saber quantos são.

```
produtos = ["Arroz", "Leite", "Pão", "Manteiga"]
```

```
for produto in produtos:
```

```
    print("🛒 Produto:", produto)
```

```
# Saída:
```

```
# 🛒 Produto: Arroz
```

```
# 🛒 Produto: Leite
```

```
# 🛒 Produto: Pão
```

```
# 🛒 Produto: Manteiga
```

Cofinanciado por:

Aplicação do for no Sistema

No sistema de encomendas, usamos o `for` para listar todos os produtos de uma encomenda de forma automática.

```
produtos = ["Teclado", "Rato", "Monitor", "Headset"]
```

```
print("📦 Produtos da encomenda:")
```

```
print("-" * 30)
```

```
for produto in produtos:
```

```
    print("  ▶", produto)
```

```
print("-" * 30)
```

```
print("Total de produtos:", len(produtos))
```

💡 A função `len()` devolve o número de elementos de uma lista. Muito útil para contagens!

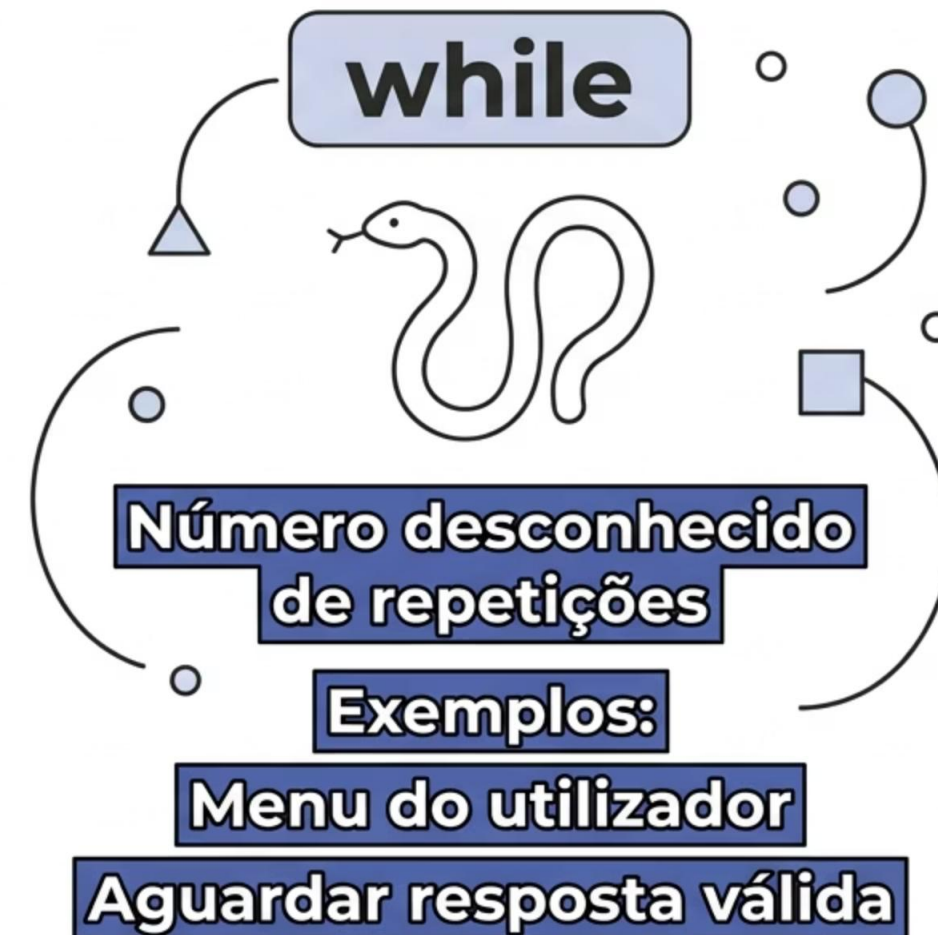
Comandado por:

while vs for — Quando Usar Cada Um?

REPETE POR SEQUÊNCIA



REPETE POR CONDIÇÃO



A escolha certa entre `while` e `for` torna o código mais claro e evita erros. Usa `while` quando dependes de uma decisão do utilizador; usa `for` quando sabes o que vais percorrer.

Cofinanciado por:

Ciclos + Condicionais: Lógica Real

Na programação real, raramente usamos uma única estrutura isolada. A verdadeira potência aparece quando combinamos ciclos com condições — é assim que se criam comportamentos inteligentes.

```
produtos = ["Teclado", "Rato", "Monitor", "Headset"]

for produto in produtos:
    if produto == "Monitor":
        print(" ★", produto, "— Produto de valor elevado")
    else:
        print(" ▶", produto, "— Produto normal")
```

while + match-case — A Base do Sistema

Esta combinação é a estrutura central de muitos programas de consola. O `while` mantém o menu ativo; o `match-case` direciona cada escolha.

```
opcao = -1

while opcao != 0:

    print("\n")

    print("┌──────────────────────────┐")
    print("│ SISTEMA DE ENCOMENDAS │")
    print("└──────────────────────────┘")

    print("│ 1 - Nova encomenda │")
    print("│ 2 - Ver produtos   │")
    print("│ 0 - Sair           │")
    print("└──────────────────────────┘")

    opcao = int(input("Opção: "))
```

```
match opcao:

    case 1:

        print("📦 A registar encomenda...")

    case 2:

        print("📋 A mostrar produtos...")

    case 0:

        print("👋 A sair do sistema...")

    case _:

        print("❌ Opção inválida!")
```

Cofinanciado por:

Funções: Organizar para Reutilizar

Uma função é um bloco de código reutilizável com um nome. Em vez de repetir o mesmo código várias vezes, defines a lógica uma vez e chamas-a sempre que precisares.

📄 🧠 Analogia: *"Uma função é como uma máquina de café — colocamos a cápsula (parâmetros), a máquina processa, e sai o café (retorno). Podes usar a máquina quantas vezes quiseres."*



Definir uma Função

Sintaxe básica:

```
def nome_da_funcao(parametros):  
    # lógica da função  
    return resultado
```

- `def` — palavra-chave para definir
- `nome_da_funcao` — nome descritivo
- `parametros` — valores de entrada
- `return` — valor devolvido

Exemplo no sistema:

```
def calcular_total(preco, quantidade):  
    return preco * quantidade
```

```
# Chamar a função  
total = calcular_total(15, 3)  
print("Total:", total) # 45
```

 A função só é executada quando a chamamos. Definir não é executar!

Parâmetros e Retorno

Os parâmetros são os "ingredientes" que entram na função. O return é o resultado que a função devolve ao código que a chamou. Sem `return`, a função executa mas não devolve nada útil.

1

Parâmetros

Valores que a função recebe ao ser chamada. Podem ser de qualquer tipo. A função pode ter 0 ou mais parâmetros.

2

Corpo da Função

O bloco de código que executa a lógica. Deve estar indentado (4 espaços ou 1 tab).

3

Return

Devolve um valor ao código que chamou a função. Termina a execução da função imediatamente.

Função de Desconto — Lógica Encapsulada

Vamos criar uma função que encapsula toda a lógica de desconto do nosso sistema. Assim, podemos reutilizá-la em qualquer parte do programa sem repetir código.

```
def aplicar_desconto(total):  
    """Aplica desconto progressivo com base no total."""  
    if total > 100:  
        desconto = 0.15  
        print("🏆 Desconto de 15% aplicado!")  
    elif total > 50:  
        desconto = 0.10  
        print("👍 Desconto de 10% aplicado.")  
    else:  
        desconto = 0  
        print("Sem desconto para este total.")  
    return total * (1 - desconto)
```

```
# Utilização  
total_com_desconto = aplicar_desconto(120)  
print("Valor final:", total_com_desconto, "€")
```

💡 As aspas triplas `"""..."""` são a docstring — documentação da função. Boa prática!

Funções para o Menu e Listagem

Agora vamos criar funções para as outras partes do sistema — mostrar o menu e listar os produtos. O código fica mais organizado e fácil de manter.

```
def mostrar_menu():  
    print("\n")  
    print("┌──────────────────┐")  
    print("│ SISTEMA DE ENCOMENDAS │")  
    print("└──────────────────┘")  
    print("│ 1 - Nova encomenda │")  
    print("│ 2 - Ver produtos   │")  
    print("│ 0 - Sair           │")  
    print("└──────────────────┘")  
    return int(input("Opção: "))
```

```
def listar_produtos(produtos):  
    print("\n📦 Produtos registados:")  
    if len(produtos) == 0:  
        print(" (sem produtos)")  
    else:  
        for produto in produtos:  
            print(" ▶", produto)
```

Exercício: Criar Funções para o Sistema

Pensa no código que já escrevemos. Quais as partes que se repetem ou que poderiam ser reutilizadas? Converte-as em funções.

Tarefa 1

Criar a função `calcular_total(preco, quantidade)` que devolve o produto dos dois valores.

Tarefa 2

Criar a função `aplicar_desconto(total)` com a lógica dos três níveis de desconto.

Tarefa 3

Criar a função `registar_encomenda()` que pede preço e quantidade ao utilizador e mostra o total final.

 Dica: começa por identificar os inputs e o output de cada função antes de escrever o código.

🔧 Sistema Completo — Versão Integrada

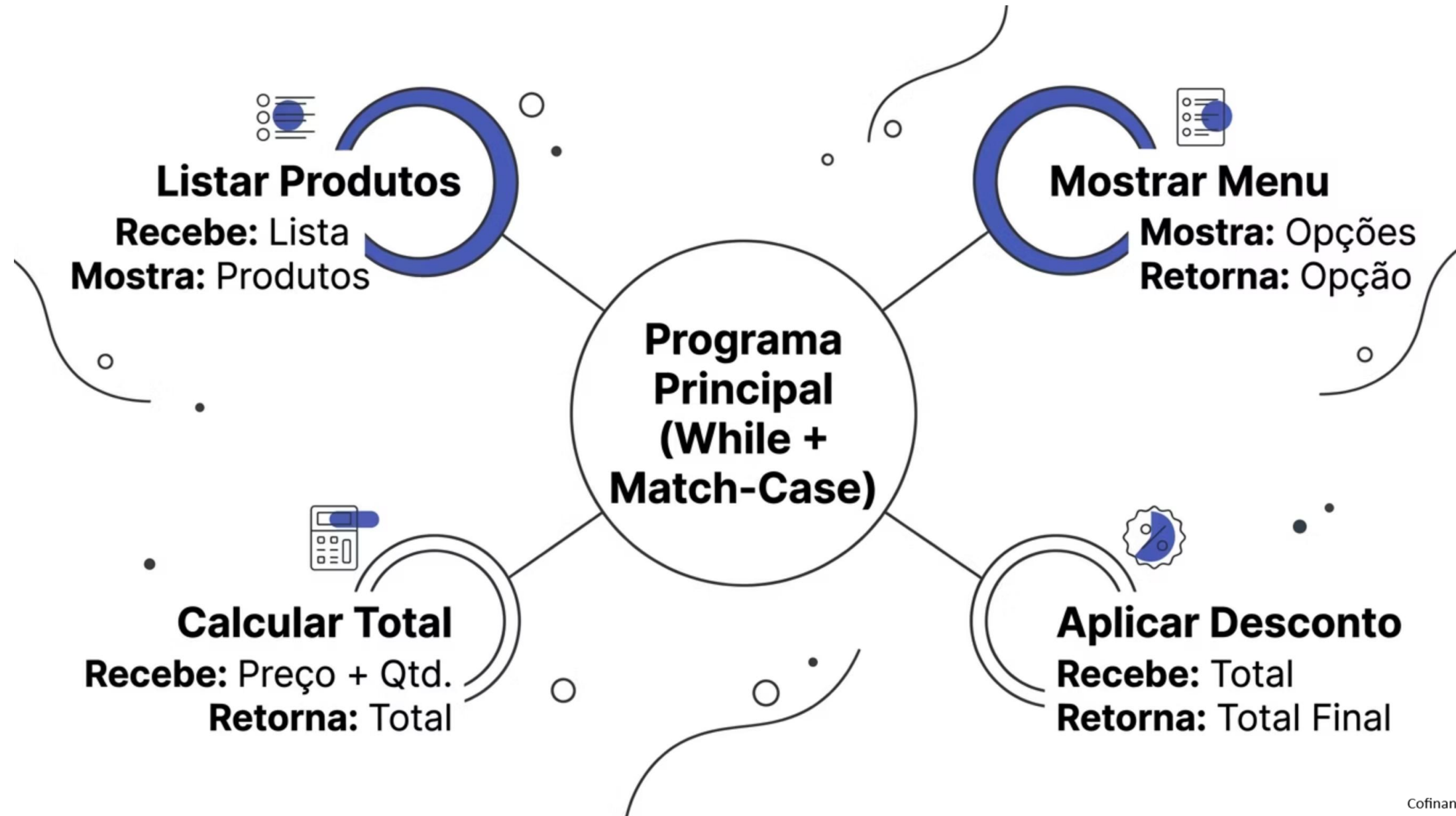
Chegou o momento de juntar tudo o que aprendemos.

Este é o sistema completo com operadores, condicionais, ciclos e funções a trabalhar em conjunto.

```
def calcular_total(preco, quantidade):  
    return preco * quantidade  
  
def aplicar_desconto(total):  
    if total > 100:  
        return total * 0.85  
    elif total > 50:  
        return total * 0.90  
    else:  
        return total  
  
def mostrar_menu():  
    print("\n1 - Nova encomenda | 2 - Ver produtos | 0 -  
Sair")  
    return int(input("Opção: "))  
  
def listar_produtos(produtos):  
    for p in produtos:  
        print(" ▶", p)  
  
produtos = []  
opcao = -1  
  
while opcao != 0:  
    opcao = mostrar_menu()  
    match opcao:  
        case 1:  
        produto = input("Produto: ")  
        preco = float(input("Preço: "))  
        qtd = int(input("Quantidade: "))  
        total = calcular_total(preco, qtd)  
        total_final = aplicar_desconto(total)  
        produtos.append(produto)  
        print(f"✅ Total final: {total_final:.2f}€")  
        case 2:  
        listar_produtos(produtos)  
        case 0:  
        print("👋 Até logo!")  
        case _:  
        print("❌ Opção inválida")
```

A Arquitetura do Sistema

Observa como cada parte do sistema corresponde a um conceito que aprendemos. A arquitetura está clara graças às funções.



Cofinanciado por:

Erros Comuns — Aprende com os Enganos

Todos os programadores cometem estes erros. Conhecê-los antecipadamente poupa muito tempo de depuração.

✗ Esquecer os dois pontos :

```
if total > 50      #  
SyntaxError  
if total > 50:    #
```

✓ Correto

✗ Usar = em vez de ==

```
if opcao = 1:      #  
SyntaxError  
if opcao == 1:    #
```

✓ Correto

✗ Loop infinito no while

```
while opcao != 0:  
    print("Menu")  
# opcao nunca muda!
```

✗ Esquecer o return

```
def calcular(a, b):  
    resultado = a * b  
    # sem return → devolve  
    None!
```

✗ Confundir while com for

```
# Listar produtos com while  
(errado)  
i = 0  
while i < len(produtos): #  
    usa for!
```

Cofinanciado por:

Exercício Autónomo: Melhorar o Sistema

Agora é a tua vez! Com base no sistema completo, adiciona as seguintes melhorias de forma autónoma:

- 1** Pedir o nome do cliente no início
Guarda numa variável e mostra-o nas mensagens de confirmação.
- 2** Pedir o nome do produto ao registar
Já está na versão completa — verifica se está a funcionar corretamente.
- 3** Validar a opção do menu
Se o utilizador introduzir um valor não numérico, o programa não deve dar erro — usa o `case _` adequadamente.
- 4** Mostrar o número de produtos registados
Na opção "Ver produtos", mostra também o total com `len()`.



Desafio Final — Eleva o Sistema

Tens os conhecimentos necessários para este desafio. Vai além do sistema base e cria uma versão mais avançada. Trabalha de forma autónoma ou em par.

Várias Encomendas

Guarda múltiplas encomendas (dicionários ou listas) e permite registar mais do que uma por sessão.

Listagem Final

Cria uma função que mostra todas as encomendas com nome do produto, preço, quantidade e total.

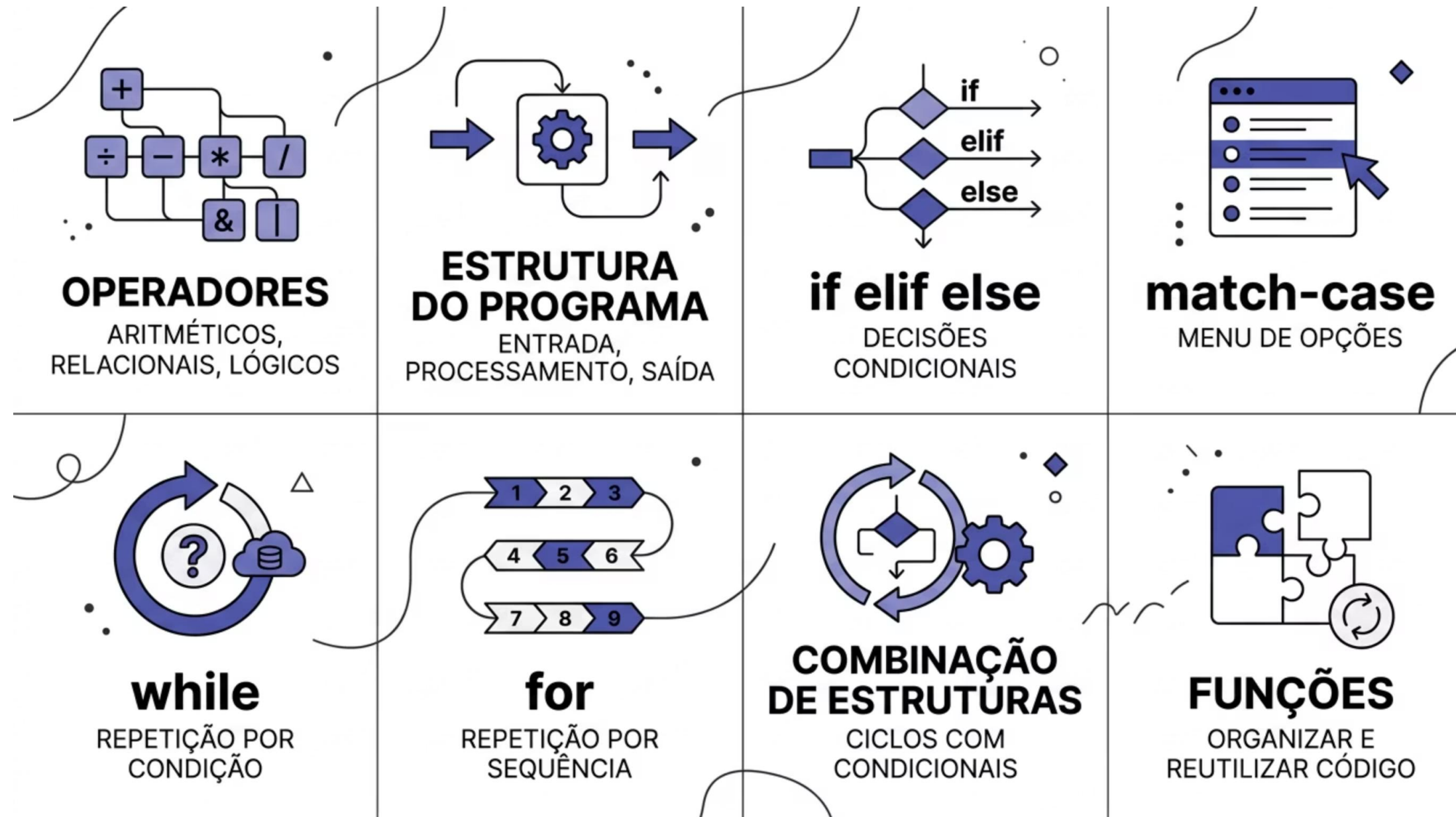
Contagem e Totais

Mostra o número total de produtos e a soma de todos os valores pagos na sessão.



Resumo da Aula

Hoje construímos um sistema real utilizando todos os conceitos da aula. Cada peça tem o seu lugar no programa que criámos juntos.



❏ ? Pergunta final à turma: "Onde apareceu cada um destes conceitos no sistema que criámos hoje?"

Cofinanciado por:

O Que Construíste Hoje

4

Funções criadas

calcular_total, aplicar_desconto, mostrar_menu, listar_produtos

2

Tipos de ciclos

while para o menu principal, for para listar produtos

3

Tipos de operadores

Aritméticos, relacionais e lógicos aplicados no sistema

1

Sistema completo

Um programa funcional de gestão de encomendas do zero

Objetivos Alcançados

→  Usas operadores corretamente

Para calcular totais, comparar valores e combinar condições lógicas.

→  Aplicas condições

if, elif, else e match-case para criar lógica de decisão real.

→  Crias e distingues ciclos

while para menus interativos, for para percorrer coleções de dados.

→  Estruturas o código com funções

Código modular, legível e reutilizável como um profissional.

→  Desenvolveste um programa completo

Do primeiro `input()` ao sistema integrado com menu e funções.



"Aprender programação faz-se programando."

Cada estrutura só ganha sentido quando é usada num problema real. Hoje fizeste exatamente isso — parabéns pelo progresso!

Continua a praticar. Experimenta modificar o sistema, adicionar funcionalidades, e quebrar o código — é assim que se aprende a programar de verdade.

Cofinanciado por: